SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 2 – Agile Sprint Planning & User Story Workshop

Course Code:	CSA10 / CSA1021	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 2 – **Agile Sprint Planning & User Story Workshop**
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	R SARVESH	**Reg. No.:**	192521143
Date:	14/10/2026	**Duration:**	60 minutes

Code of Conduct

I R SARVESH / 192521143 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: R SARVESH

Annexure A – Agile Sprint Planning & User Story Workshop

Instructions to Students:

Each student is assigned an individual software project problem statement. Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

Based on the given problem statement, perform an **Agile Sprint Planning and User Story Workshop** by applying Agile software engineering practices. Prepare a comprehensive sprint planning document covering the following aspects:

1. **Identify the Project Vision and Stakeholders**
   - Define the project's vision, objectives, and identify all key stakeholders involved.
2. **Create Epics and User Stories**
   - Break down the requirements into Epics and formulate well-structured User Stories using the format: *As a <user>, I want <feature>, so that <benefit>*.
3. **Define Acceptance Criteria**
   - Write clear and measurable acceptance criteria for each selected user story using the Given–When–Then format or equivalent.
4. **Prioritize the Product Backlog**
   - Organize and prioritize the product backlog based on business value, customer needs, and project dependencies.
5. **Plan the Sprint Backlog**
   - Select high-priority user stories for the first sprint and divide them into actionable development tasks.
6. **Estimate Effort Using Story Points**
   - Assign story points to each selected user story using an appropriate estimation technique (e.g., Fibonacci sequence or Planning Poker) and justify your estimates.
7. **Present the Sprint Goal and Expected Deliverables**
   - Define a clear sprint goal and describe the expected sprint deliverables, including the features to be completed at the end of the sprint.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Project Vision & Stakeholder Identification (30%)	Project vision is clear, comprehensive, and aligned with the problem statement. All relevant stakeholders and their roles are correctly identified.	Vision is clear with minor omissions. Most stakeholders are correctly identified.	Vision is partially defined. Some stakeholders are missing or incorrectly identified.	Vision is unclear or incomplete. Stakeholders are poorly identified or missing.
Epics & User Stories (25%)	Epics are well-defined and user stories are complete, follow Agile format, and fully capture functional requirements.	Epics and user stories are mostly complete with minor formatting or requirement issues.	User stories are partially complete with limited Agile structure.	User stories are incomplete, unclear, or do not follow Agile practices.
Acceptance Criteria (20%)	Acceptance criteria are specific, measurable, testable, and aligned with all user stories.	Acceptance criteria are mostly clear and testable with minor gaps.	Acceptance criteria are partially defined and lack clarity or completeness.	Acceptance criteria are missing or not aligned with user stories.
Product Backlog Prioritization & Sprint Planning (15%)	Product backlog is logically prioritized, sprint backlog is realistic, and task breakdown is complete.	Backlog prioritization and sprint planning are mostly appropriate with minor issues.	Prioritization or sprint planning is partially complete with limited justification.	Backlog prioritization and sprint planning are poorly organized or incomplete.
Story Point Estimation, Sprint Goal & Presentation (10%)	Story point estimation is well justified, sprint goal is clear, deliverables are realistic, and presentation is professional and well organized.	Estimation and sprint goal are mostly appropriate with minor inconsistencies. Presentation is clear.	Estimation or sprint goal lacks justification. Presentation is adequate but incomplete.	Estimation is unrealistic or missing. Sprint goal, deliverables, and presentation are unclear or incomplete.

Criteria	Mark
Project Vision & Stakeholder Identification **(30%)**	/5
Epics & User Stories **(25%)**	/5
Acceptance Criteria **(20%)**	/5
Product Backlog Prioritization & Sprint Planning **(15%)**	/5
Story Point Estimation, Sprint Goal & Presentation **(10%)**	/5
TOTAL	/25

1. Autonomous Drone-Carrier Mesh Network for Hyper-Local Medical Logistics

Industry Context

Modern municipal emergency response infrastructures are heavily bottlenecked by urban traffic congestion, grid design limits, and unpredictable weather anomalies. During critical medical crises—such as sudden cardiac arrests, acute anaphylaxis, or postpartum hemorrhages—the delivery time of life-saving medical supplies (e.g., Automated External Defibrillators (AEDs), EpiPens, rare blood types, or anti-venoms) determines patient survival. Traditional ambulance response times in dense metropolises often stretch beyond the critical 4-to-6-minute survival window.

While commercial drone delivery networks are emerging, individual drone units suffer from limited operational ranges, rapid battery depletion, and high vulnerability to sudden micro-climate shifts. Centralized cloud-based dispatch systems introduce single points of failure and high network latency penalties, which are unacceptable in life-or-death scenarios. There is an immediate structural demand for a decentralized, edge-computed logistics platform that treats automated delivery vehicles, municipal rooftops, and public transit nodes as a cohesive, self-healing transit mesh network.

Problem Statement

Design and develop a highly concurrent, fault-tolerant Autonomous Drone-Carrier Mesh Network that dynamically routes, schedules, and dispatches automated medical delivery drones across dynamic smart city topographies.

The platform must ingest high-velocity streaming telemetry—including real-time 3D drone coordinates, localized wind velocity vectors, active airspace geofences, battery health diagnostics, and regional cellular signal strengths—from hundreds of airborne units and municipal charging hubs concurrently.

Using decentralized routing engines, the platform must coordinate drone handovers between automated charging platforms mounted on moving public transit buses and static municipal smart buildings. The solution must run autonomous collision avoidance, handle sudden edge computing node dropouts, and guarantee zero-latency emergency rerouting without relying on continuous, active cloud infrastructure connectivity.

Objectives

The platform should:

1. **Ingest and Process Sub-Second Telemetry:** Capture high-frequency 3D spatial positions and hardware diagnostic data from hundreds of active airborne drone arrays without packet loss.
2. **Orchestrate Dynamic 4D Routing:** Compute optimal, collision-free 4D flight corridors (latitude, longitude, altitude, time) that adapt dynamically to shifting wind patterns and temporary no-fly zones.

3. **Manage Moving Node Handovers:** Coordinate micro-second landing and deployment schedules onto mobile public transit carrier platforms using localized edge mesh communication loops.
4. **Enforce Edge Autonomy & Self-Healing:** Implement decentralized routing protocols that allow the local swarm to self-heal and re-allocate tasks autonomously if the central cloud goes offline.
5. **Ensure Mission-Critical Prioritization:** Design a strict traffic prioritization engine that automatically clears airspace and throttles commercial data streams when a high-priority medical alert is triggered.
6. **Generate Cryptographic Provenance Audits:** Maintain unalterable, cryptographically signed operational logs to satisfy civil aviation authorities and medical privacy compliance boards.

Expected Outcomes

The proposed platform will:

1. **Minimize Life-Saving Delivery Windows:** Reduce the average municipal transit window for critical medical payloads to under 4 minutes.
2. **Extend Fleet Operational Reach:** Increase effective drone flight ranges by up to 300% through optimized, multi-stage mobile carrier handovers.
3. **Guarantee High System Availability:** Achieve 99.999% network uptime for emergency dispatch tracking via fully decentralized edge computing worker networks.
4. **Lower Operational Asset Losses:** Eliminate mid-air drone collisions and forced emergency landings via highly responsive, predictive collision avoidance networks.
5. **Provide Regulatory Transparency:** Enable seamless compliance tracking for aviation authorities through instantly generated, immutable flight performance audit trails.

Agile Sprint Planning and User Story Workshop

1. Project Vision and Stakeholders

1.1 Project Vision

The vision is to develop a decentralized, fault-tolerant medical logistics network that uses autonomous drones, municipal rooftops, moving public-transit buses, and smart buildings to deliver emergency medical supplies rapidly and safely.

The system will process real-time telemetry, dynamically calculate routes, coordinate transfers between drones and carrier platforms, and continue essential local operations during central cloud failures. It will also prioritize emergency missions and maintain cryptographically verifiable mission records.

1.2 Project Objectives

1. Rapid delivery: Target an average delivery time of under four minutes for critical medical missions.
2. Dynamic routing: Calculate routes using latitude, longitude, altitude, time, wind conditions, and restricted airspace.
3. Real-time monitoring: Process telemetry from drones and carrier platforms with minimal delay and without avoidable packet loss.
4. Platform handover: Transfer missions between drones, buses, charging points, and suitable stationary platforms.
5. Fault tolerance: Preserve supported local operations when central cloud connectivity is lost.
6. Emergency prioritization: Give critical medical missions priority over commercial workloads while enforcing safety constraints.
7. Secure auditing: Maintain signed, tamper-evident records for mission verification, accountability, and privacy compliance.

These are design objectives and proposed performance targets. They must be validated through simulation, system testing, and controlled operational trials.

1.3 Project Scope

Included in Scope	Outside the Initial Sprint Scope
Emergency request registration	Full autonomous flight deployment
Telemetry ingestion and monitoring	City-wide production deployment
Dynamic routing and geofence validation	Unrestricted autonomous operation
Emergency task prioritization	Final aviation certification
Edge-node fault recovery	Guaranteed real-world delivery times
Platform handover coordination	Full integration with every transport operator
Cryptographic audit records	Complete regulatory approval

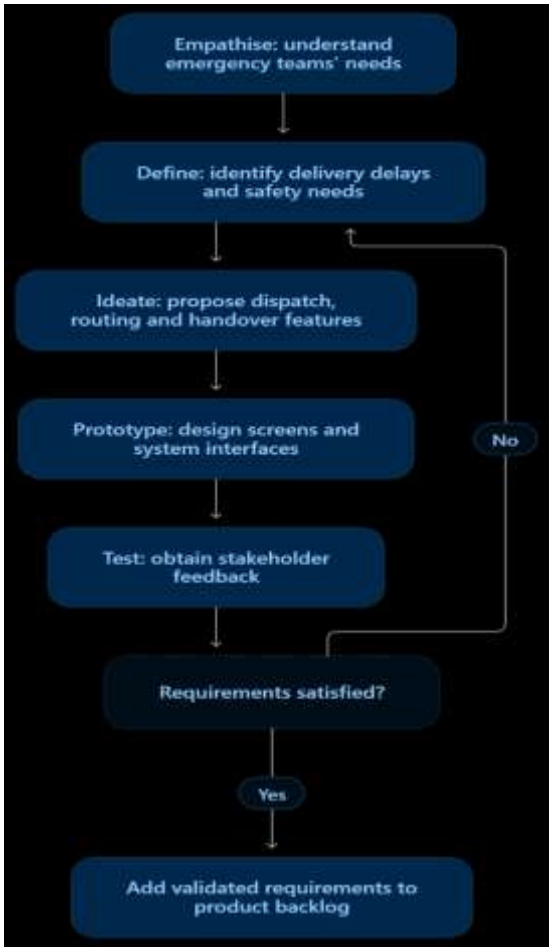
The initial development will establish a testable software foundation before progressing to advanced routing, distributed coordination, and physical deployment.

1.4 Stakeholders and Responsibilities

Stakeholder	Responsibilities
Emergency medical teams	Request supplies and communicate urgency and delivery details
Patients and recipients	Receive medical supplies and benefit from faster response
Drone operators	Monitor mission progress and intervene when required
Municipal and aviation authorities	Define airspace restrictions and operational requirements
Medical suppliers	Prepare, verify, and hand over medical packages
Product Owner	Prioritize the product backlog according to value and risk
Scrum Master	Facilitate Scrum events and remove process obstacles
Development team	Design, implement, integrate, and test software
Security and compliance team	Protect sensitive data and validate audit integrity
Bus, rooftop, and building platform operators	Provide eligible handover, charging, or coordination facilities

1.5 User-Centred Design Process

The lecture notes describe Design Thinking as an iterative process of empathizing, defining, ideating, prototyping, and testing.



This process helps ensure that features are derived from actual emergency logistics needs rather than technical assumptions alone.

2. Epics and User Stories

An epic represents a major area of functionality. A user story expresses a smaller requirement from the perspective of a user or system actor.

The standard format is:

As a [user], I want [feature], so that [benefit].

2.1 Epic and User Story Catalogue

Epic ID	Epic	User Story
E1	Emergency Dispatch	US-01: As an emergency operator, I want to create a medical delivery request so that urgent supplies can be dispatched quickly.
E2	Telemetry Monitoring	US-02: As a drone operator, I want to monitor drone location, battery level, and wind conditions so that I can assess operational status.
E3	Dynamic Routing	US-03: As the routing system, I want to recalculate routes when wind conditions

		or restricted zones change so that drones can follow validated safe routes.
E4	Platform Handover	US-04: As a drone, I want to transfer a mission to an eligible receiving platform so that delivery range can be extended.
E5	Fault Tolerance	US-05: As a system administrator, I want edge nodes to continue supported local operations during cloud failure so that essential missions are not unnecessarily interrupted.
E6	Emergency Prioritization	US-06: As an emergency operator, I want critical medical missions to receive priority so that life-saving deliveries are not delayed by commercial workloads.
E7	Secure Auditing	US-07: As a compliance officer, I want signed, tamper-evident mission records so that mission activities can be verified.

2.2 Story Decomposition

Each story must be small enough to estimate, implement, and test within a sprint. For example:



The same decomposition process will be applied to telemetry, routing, handover, fault recovery, prioritization, and audit features when they enter their respective sprints.

3. Acceptance Criteria for User Stories

Acceptance criteria define the conditions that must be satisfied before a story can be accepted. They must be specific, observable, and testable.

The Given–When–Then format consists of:

- Given: The initial condition or context.
- When: The action or event.
- Then: The expected system response.

3.1 Acceptance Criteria Catalogue

US-01: Emergency Dispatch

- Given an authorized operator and a valid medical delivery request, when the operator submits the request, then the system validates the required fields and creates a unique mission ID.
- Given a request with missing mandatory fields, when it is submitted, then the system rejects it and identifies the validation errors.
- Given a successfully created mission, when the operator opens its details, then the recorded priority and mission status are displayed.

US-02: Telemetry Monitoring

- Given a connected drone transmitting telemetry, when the system receives a valid telemetry message, then it validates and records the location, battery level, wind data, and timestamp.
- Given malformed or incomplete telemetry, when the message is processed, then it is rejected or quarantined and the event is logged.
- Given a valid update, when the monitoring interface refreshes, then the latest accepted status is displayed.

US-03: Dynamic Routing

- Given a planned route and valid environmental data, when a wind change or restricted zone affects that route, then the routing service evaluates an alternative route.
- Given a proposed alternative route, when safety and geofence validation fails, then the system must not authorize that route.
- Given no safe alternative can be established, when the route becomes unusable, then the system raises an exception requiring an approved safe response.

US-04: Platform Handover

- Given a drone approaching an eligible receiving platform, when the handover process begins, then the system verifies platform availability and compatibility.
- Given a receiving platform that confirms readiness, when the handover is committed, then mission ownership and status are updated consistently.
- Given a failed handover, when confirmation is not received, then the mission remains recoverable and the system records the failure.

US-05: Fault Tolerance

- Given an edge node operating with approved local safety rules, when central cloud connectivity is lost, then supported local functions continue without requiring a cloud response.
- Given that the cloud connection is restored, when synchronization begins, then mission records are reconciled without silently overwriting conflicting data.

- Given a task that requires unavailable information or authority, when the system cannot verify safe execution, then it must not proceed merely to maintain availability.

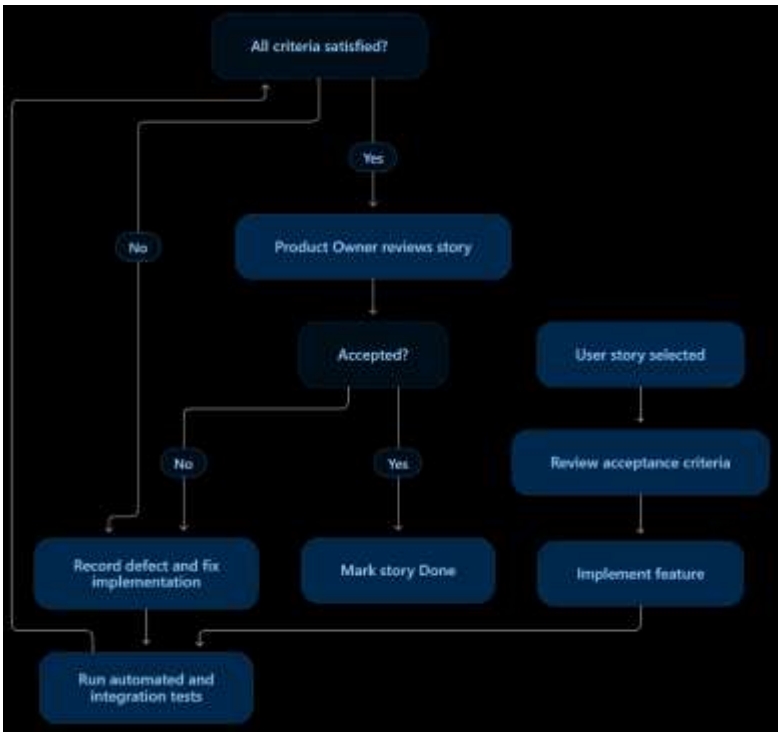
US-06: Emergency Prioritization

- Given emergency and commercial tasks competing for limited resources, when scheduling occurs, then eligible emergency tasks receive the configured priority.
- Given a high-priority mission, when the scheduler assigns resources, then airspace restrictions and collision-avoidance requirements remain mandatory.
- Given insufficient resources for a mission, when it cannot be safely scheduled, then the system reports the constraint rather than falsely confirming dispatch.

US-07: Secure Auditing

- Given a mission event requiring an audit record, when the event is recorded, then the system generates a signed record containing the required mission reference, timestamp, and event details.
- Given a valid audit record, when an authorized verifier checks its signature and integrity, then the verification result is returned.
- Given a record that has been altered or has an invalid signature, when verification is performed, then the system identifies the integrity failure.

3.2 Acceptance Testing Flow



Acceptance testing must demonstrate the defined behaviour. A story is not considered complete merely because its code has been written.

4. Product Backlog Prioritization

The product backlog is the ordered collection of requirements, enhancements, fixes, and technical work needed to develop the product.

Items are prioritized according to:

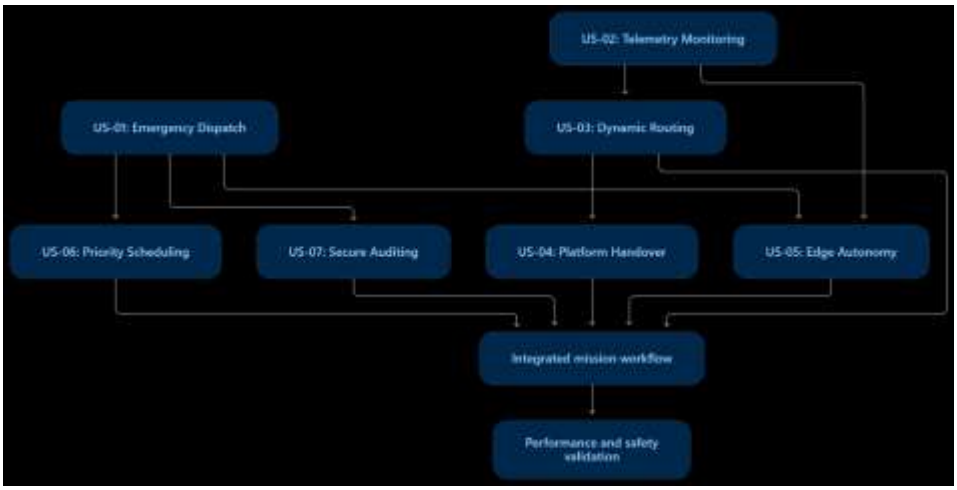
1. Business value: Contribution to faster, safer medical delivery.
2. Customer need: Importance to emergency teams and recipients.
3. Dependencies: Whether other features require the item first.
4. Risk: Potential safety, reliability, privacy, or security impact.
5. Uncertainty: Need to validate a technically complex assumption.

4.1 Prioritized Product Backlog

Rank	ID	Backlog Item	Value	Dependency
1	US-01	Emergency request registration	High	None
2	US-02	Telemetry ingestion and monitoring	High	Telemetry interface
3	US-06	Emergency priority scheduling	Very high	Request and task data
4	US-03	Dynamic routing and geofence validation	Very high	Telemetry and map data
5	US-07	Signed mission audit records	High	Mission events and key management
6	US-05	Edge autonomy and recovery	Very high	Local state and safety rules
7	US-04	Drone-to-platform handover	High	Routing and platform availability
8	NFR-01	Load, reliability, and latency testing	High	Working system increments
9	NFR-02	Security and privacy verification	High	Identity, data flows, and access controls

The ranking is a proposed initial order. Safety-critical risks can justify moving security, fault-tolerance, or routing work earlier after technical review.

4.2 Dependency Diagram



Prioritization decision: Dispatch and telemetry form the initial foundation. Routing and handover depend on trustworthy mission and operational data. Fault tolerance and security must be designed from the beginning, even when their full implementation occurs in later sprints.

4.3 Definition of Product Backlog Readiness

Before a story is selected for a sprint, it should have:

- A clear user or system need.
- A defined business benefit.
- Acceptance criteria.
- Identified dependencies and risks.
- An agreed estimate or an explicit plan to resolve uncertainty.
- Sufficient detail for the development team to begin work.

This prevents ambiguous requirements from entering implementation without adequate planning.

5. Sprint Planning and Sprint Backlog

Sprint Planning converts selected product backlog items into a realistic short-term delivery commitment. The team defines a goal, selects stories based on priority and capacity, and identifies actionable tasks.

5.1 Sprint 1 Plan

Duration: Two weeks Sprint Goal: Establish a tested foundation that allows emergency operators to create medical delivery requests and monitor incoming drone telemetry.

Sprint 1 selected stories: US-01 and US-02.

The team should confirm its actual availability before committing to these stories. The estimates below are planning assumptions, not measured team velocity.

Sprint 1 Task Breakdown

Story	Task ID	Actionable Task	Completion Evidence
US-01	T1.1	Define request fields, data types, mandatory values, and validation rules	Approved request schema
US-01	T1.2	Design the mission creation API and error responses	API specification
US-01	T1.3	Implement request validation and unique mission ID generation	Working endpoint
US-01	T1.4	Store accepted mission details and initial status	Persisted mission record
US-01	T1.5	Write tests for valid, invalid, and incomplete requests	Passing test results
US-02	T2.1	Define the telemetry message schema and timestamp rules	Approved telemetry schema
US-02	T2.2	Implement the telemetry ingestion endpoint or message consumer	Working ingestion component
US-02	T2.3	Validate telemetry fields and handle malformed messages	Validation tests
US-02	T2.4	Display the latest accepted drone status in a monitoring view	Demonstrable interface
US-02	T2.5	Test valid messages, invalid messages, and repeated updates	Test results and defect log
Foundation	T3.1	Establish source control, development setup, and CI checks	Reproducible setup
Foundation	T3.2	Define module interfaces and basic integration tests	Reviewed interfaces and test results

5.2 Sprint Workflow



The team reviews progress regularly. If a story becomes too large or blocked, the team discusses scope and dependencies with the Product Owner rather than silently compromising acceptance criteria.

5.3 Sprint 1 Deliverables

At the end of Sprint 1, the team is expected to demonstrate:

1. An emergency request can be submitted and validated.
2. A unique mission ID and initial status are created.
3. Valid telemetry can be ingested and displayed.
4. Invalid inputs are handled appropriately.
5. Automated tests provide evidence for the accepted behaviour.
6. The source code and development setup are available for continued work.
7. Unfinished tasks, known defects, and follow-up stories are recorded in the backlog.

These deliverables represent an initial increment, not a complete autonomous drone logistics system.

6. Story Point Estimation and Planning Poker

Story points are relative estimates of effort, complexity, uncertainty, and testing requirements. They are not exact development hours.

6.1 Fibonacci Estimation Scale

Points	Meaning	Typical Work
1	Very small	Simple validation or configuration
2	Small	Limited change with low uncertainty
3	Moderate	Standard feature with a few tests
5	Medium–large	Multiple components or integration
8	Large	Complex coordination or recovery logic
13	Very large	High uncertainty or safety-critical complexity

If a story is estimated at 13 points, the team should consider breaking it into smaller stories or conducting a technical investigation before committing to implementation.

6.2 Proposed Story Estimates

Story	Points	Justification
US-01: Emergency Dispatch	3	Request validation, mission ID generation, and persistence
US-02: Telemetry Monitoring	5	Message ingestion, validation, and status display
US-03: Dynamic Routing	13	Route calculation, environmental changes, geofence constraints, and safety testing
US-04: Platform Handover	8	Platform availability, mission transfer, and failure recovery
US-05: Fault Tolerance	13	Distributed state, local autonomy, and recovery synchronization
US-06: Emergency Prioritization	5	Scheduling rules, priority handling, and validation
US-07: Secure Auditing	8	Cryptographic signatures, verification, and tamper detection

6.3 Planning Poker Procedure

1. The Product Owner explains the story, its value, and its acceptance criteria.
2. Team members clarify requirements, risks, and dependencies.
3. Each member privately selects a Fibonacci estimate.
4. Estimates are revealed simultaneously.

5. Members with different estimates explain their reasoning.
6. The team discusses uncertainty and reaches an agreed estimate.
7. The estimate is recorded and revised if new information changes the scope.

The team should use actual completed work to understand its capacity for future sprints rather than assuming a fixed velocity from the beginning.

7. Sprint Goal, Definition of Done, and Evaluation

7.1 Sprint Goal

To deliver a tested foundation for emergency dispatch and drone telemetry monitoring, enabling valid medical requests to be registered and operational data to be received and displayed.

This goal focuses on an integrated, demonstrable increment rather than completing unrelated tasks.

7.2 Sprint Review

The Sprint Review demonstrates the working increment to stakeholders. The team should:

- Submit a valid emergency request.
- Demonstrate mission ID generation and initial status.
- Send sample telemetry and display the latest accepted values.
- Demonstrate rejection of invalid inputs.
- Review test results, defects, and unfinished work.
- Collect stakeholder feedback and update the product backlog.

7.3 Sprint Retrospective

The team evaluates its development process.

Discussion Area	**Example Question**	**Expected Action**
What worked well?	Were the requirements sufficiently clear?	Retain effective practices
What caused problems?	Were integration tasks delayed?	Identify the root cause
What should improve?	Did testing begin too late?	Introduce earlier testing
What action will be taken?	Who will improve the test pipeline?	Assign an owner and deadline

8. Feasibility, Risks, and Ethical Requirements

8.1 Feasibility Analysis

Feasibility Type	Assessment Required
Technical	Confirm that telemetry ingestion, edge computing, secure communications, routing, and platform integration are technically achievable.
Operational	Evaluate emergency workflows, operator training, platform availability, and regulatory constraints.
Financial	Estimate development, drone hardware, charging infrastructure, connectivity, maintenance, and testing costs.

8.2 Major Risks and Mitigation

Risk	Potential Impact	Mitigation
Sudden weather changes	Route disruption	Continuously validate environmental data and alternative routes
Network or cloud failure	Delayed coordination	Provide tested local autonomy and recovery procedures
Incorrect telemetry	Unsafe decisions	Validate data, timestamps, and source authenticity
Handover failure	Mission interruption	Require receiving-platform confirmation and recovery logic
Cyberattack or forged messages	Unauthorized control or data exposure	Use authenticated communication, access controls, and key management
Conflicting emergency requests	Resource contention	Apply explicit priority policies and safety constraints
Regulatory restrictions	Deployment delays	Engage relevant authorities and validate operating requirements

8.3 Ethics, Privacy, and Security

Accordingly, the system must:

- Collect only telemetry and personal information necessary for mission execution.
- Restrict access to patient and delivery information by role.
- Encrypt sensitive data in transit and, where appropriate, at rest.
- Authenticate drones, edge nodes, and authorized operators.
- Sign and verify important mission records.

- Avoid allowing emergency priority to override collision avoidance or airspace restrictions.
- Define data retention, incident response, and audit access policies.

A tamper-evident audit mechanism supports verification, but it does not by itself guarantee that the original data was truthful or that every privacy obligation has been satisfied.

9. Expected Outcomes and Future Sprints

9.1 Expected Outcomes

The project aims to establish a reliable software foundation and progressively develop a distributed medical delivery network with:

- Faster emergency dispatch and improved mission visibility.
- Dynamic route planning and airspace validation.
- Handover between compatible drones and carrier platforms.
- Continued supported operations during cloud connectivity loss.
- Priority scheduling for urgent medical missions.
- Verifiable mission records and stronger operational accountability.

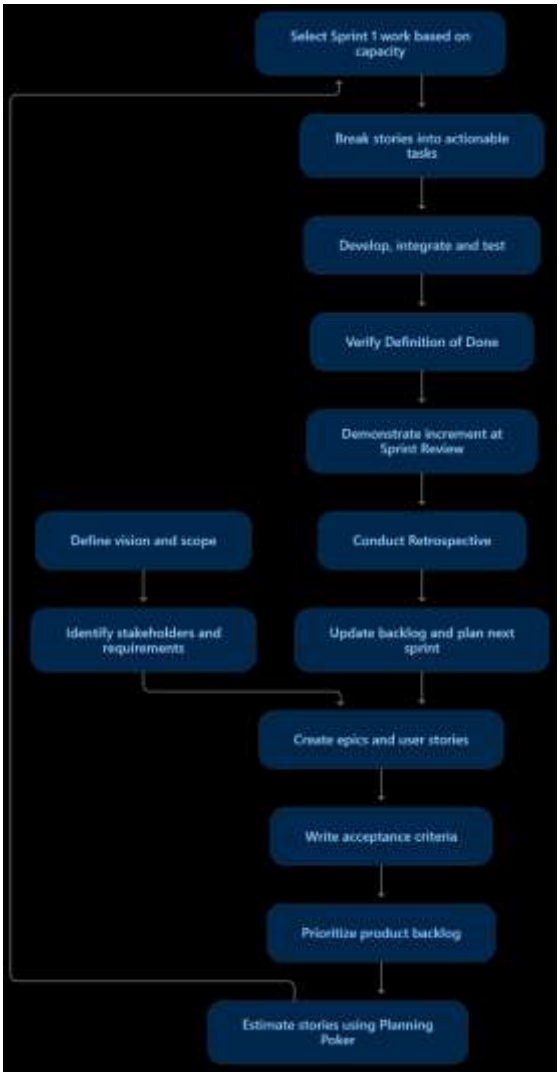
The proposed average delivery time of under four minutes, increased effective range, and 99.999% availability are targets, not demonstrated results. They require measurable test plans and sufficient evidence before being claimed as achieved.

9.2 Indicative Future Sprint Sequence

Sprint	Main Focus	Expected Increment
Sprint 1	Emergency dispatch and telemetry	Valid requests and basic monitoring
Sprint 2	Priority scheduling and mission state	Prioritized task queue
Sprint 3	Dynamic routing and geofence checks	Validated route-planning service
Sprint 4	Edge autonomy and recovery	Local operation during simulated cloud failure
Sprint 5	Platform handover	Tested mission transfer between simulated platforms
Sprint 6	Secure auditing and integration	Signed mission records and integrated workflow
Sprint 7	Performance and safety validation	Load-test results, failure tests, and defect remediation

This sequence is indicative. Backlog refinement, technical dependencies, testing results, and discovered safety risks may change the order or duration of later sprints.

10.3 Final Development Workflow



Conclusion

This Agile Sprint Planning and User Story Workshop converts the autonomous drone-carrier medical logistics problem into an actionable software development plan. It establishes a clear vision, defines stakeholders and requirements, creates testable user stories, prioritizes dependencies, and provides a structured Sprint 1 plan with estimates and deliverables.

The project will progress through incremental, tested releases, with stakeholder feedback and continuous improvement guiding subsequent sprints. Safety, security, privacy, and feasibility must remain part of development from the outset, while real-world performance claims must be supported by evidence before deployment.